HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
VIỆN KHOA HỌC KỸ THUẬT BƯU ĐIỆN

BÁO CÁO NỘI DUNG

Nội dung 1: Nghiên cứu tổng quan công nghệ STT, LLM và giao thức tích hợp

ĐỀ TÀI NGHIÊN CỨU KHOA HỌC CỦA SINH VIÊN NĂM 2026

THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG MEET-ASSISTANT: TỰ ĐỘNG HÓA TAKE-NOTE VÀ ĐỒNG BỘ HÓA QUẢN LÝ CÔNG VIỆC

18-SV-2026-RIPT

Thuộc nhóm ngành khoa học: Công nghệ thông tin

Chủ trì đề tài: Trần Đức Định (B23DCCC038)
Thành viên: Hoàng Nguyễn Trà My (B23DCCC116)
Giảng viên hướng dẫn: TS. Lê Văn Hoàng

Hà Nội – 2026

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU NỘI DUNG.....1
1.1. Nhiệm vụ theo đề cương.....1
1.2. Phương pháp thực hiện.....1
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ.....2
2.1. Bài toán trợ lý cuộc họp.....2
2.1.1. Phát biểu bài toán.....2
2.1.2. Đặc thù của hội thoại cuộc họp.....2
2.1.3. Ngữ liệu và hướng nghiên cứu liên quan.....2
2.1.4. Các giải pháp trợ lý cuộc họp hiện có.....3
2.2. Nhận dạng tiếng nói tự động.....3
2.2.1. Các hướng tiếp cận.....3
2.2.2. Mô hình Whisper.....4
2.2.3. faster-whisper, CTranslate2 và lượng tử hoá int8.....5
2.2.4. Nhận diện ngôn ngữ và hiện tượng chuyển mã Việt–Anh.....6
2.2.5. Hiện tượng ảo giác của Whisper và cách hạn chế.....6
2.2.6. Độ đo đánh giá nhận dạng tiếng nói.....6
2.3. Tách người nói.....7
2.3.1. Định nghĩa bài toán và độ đo DER.....7
2.3.2. Vector đặc trưng giọng nói.....7
2.3.3. Gom cụm phân cấp theo độ tương đồng cosine.....8
2.3.4. Vấn đề nói chồng.....9
2.4. Mô hình ngôn ngữ lớn trong trích xuất thông tin và hỏi đáp.....9
2.4.1. Kiến trúc Transformer và mô hình ngôn ngữ lớn.....9
2.4.2. Mô hình suy luận.....9
2.4.3. Thiết kế prompt và đầu ra JSON có cấu trúc.....10
2.4.4. Ảo giác của LLM, đối chiếu bằng chứng và con người trong vòng lặp.....10
2.4.5. Sinh văn bản tăng cường truy xuất.....11
2.4.6. Dịch vụ suy luận và chủ quyền dữ liệu.....12
2.5. Kiến trúc microservices và hướng sự kiện.....12
2.5.1. Microservices và cổng API.....12
2.5.2. Giao tiếp bất đồng bộ qua RabbitMQ.....13
2.5.3. Ngữ nghĩa giao nhận và tính lũy đẳng.....13
2.5.4. Mẫu transactional outbox.....13
2.6. Thu nhận nội dung họp trực tuyến.....14
2.6.1. WebRTC và xử lý âm thanh trên trình duyệt.....14
2.6.2. Tự động hoá trình duyệt với Playwright và Xvfb.....14
2.6.3. Tiện ích mở rộng Chrome Manifest V3.....14
2.7. Tích hợp nền tảng quản lý công việc.....15
2.7.1. OAuth 2.0 và Webhook.....15

2.7.2. Jira, Trello và Telegram.....15
2.8. Kiểm soát truy cập.....16
2.8.1. Mô hình RBAC.....16
2.8.2. Mô hình ABAC.....16
2.8.3. Phân quyền theo vai trò kết hợp phạm vi tài nguyên.....16
2.9. Kết luận chương.....17
CHƯƠNG 3. KẾT QUẢ NỘI DUNG VÀ ĐÁNH GIÁ.....18
3.1. Các lựa chọn công nghệ rút ra từ tổng quan.....18
3.2. Đối chiếu với kết quả cần đạt.....19
TÀI LIỆU THAM KHẢO.....20

DANH MỤC BẢNG BIỂU

Bảng 2.1. So sánh các giải pháp trợ lý cuộc họp tiêu biểu.....3
Bảng 2.2. So sánh các hướng tiếp cận nhận dạng tiếng nói.....4
Bảng 2.3. Đặc điểm các nền tảng tích hợp được khảo sát.....16
Bảng 2.4. So sánh các mô hình kiểm soát truy cập.....17
Bảng 3.1. Công nghệ chủ đạo được xác định sau bước tổng quan.....18
Bảng 3.2. Đối chiếu kết quả Nội dung 1 với đề cương.....19

DANH MỤC HÌNH VẼ

Hình 2.1. Kiến trúc mô hình Whisper với chuỗi token đặc biệt điều khiển ngôn ngữ, nhiệm vụ và mốc thời gian..... 5
Hình 2.2. Quy trình tách người nói dựa trên vector đặc trưng giọng và gom cụm phân cấp 8
Hình 2.3. Trích xuất thông tin có cấu trúc bằng LLM, đối chiếu bằng chứng và duyệt bởi con người..... 11

CHƯƠNG 1. GIỚI THIỆU NỘI DUNG

1.1. Nhiệm vụ theo đề cương

Nội dung 1 là bước khởi đầu của đề tài "Thiết kế và triển khai hệ thống Meet-Assistant: Tự động hóa take-note và đồng bộ hóa quản lý công việc", thực hiện trong tháng 05/2026. Theo đề cương đã được phê duyệt, nhiệm vụ của bước này là nghiên cứu tổng quan các công nghệ nhận dạng tiếng nói (Speech-to-Text, STT) như Whisper và wav2vec 2.0, các mô hình ngôn ngữ lớn (Large Language Model, LLM) như GPT và Gemini, cùng các giao thức tích hợp ứng dụng (Webhook, OAuth 2.0). Kết quả cần đạt gồm bản tổng quan tài liệu, xác định mô hình AI chủ đạo cho hệ thống và xác định các nền tảng quản lý công việc mục tiêu (Jira, Trello).

Bước tổng quan có vai trò định hướng cho toàn bộ các bước sau. Lựa chọn mô hình STT quyết định chất lượng transcript, mà transcript lại là đầu vào duy nhất của bước tóm tắt và trích xuất công việc. Lựa chọn kiến trúc giao tiếp giữa các thành phần quyết định khả năng thay thế mô hình AI về sau mà không phải viết lại luồng nghiệp vụ. Vì vậy, ngoài các công nghệ AI, nhóm nghiên cứu khảo sát thêm kiến trúc microservices hướng sự kiện, các cách thu nhận nội dung họp trực tuyến và các mô hình kiểm soát truy cập.

1.2. Phương pháp thực hiện

Nhóm sử dụng phương pháp nghiên cứu tài liệu: đọc các bài báo gốc về Whisper, wav2vec 2.0, x-vector, ERes2Net, Transformer và các báo cáo kỹ thuật của các họ LLM; đọc tài liệu chính thức của RabbitMQ, Chrome Extensions, Playwright, Jira, Trello và Telegram; khảo sát các sản phẩm trợ lý cuộc họp đang có trên thị trường (Otter.ai, Fireflies.ai, Tactiq, Microsoft Teams Premium) theo hai tiêu chí là cách thu nhận nội dung cuộc họp và mức độ phù hợp với nhóm nhỏ tại Việt Nam. Bên cạnh đó, nhóm chạy thử nhanh các thư viện ứng viên trên máy cá nhân để kiểm tra khả năng chạy trên CPU, vì đây là điều kiện phần cứng thực tế của nhóm.

Phần còn lại của báo cáo này trình bày kết quả tổng quan (chương tiếp theo) và các lựa chọn công nghệ rút ra từ tổng quan (chương cuối).

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ

Chương này trình bày nền tảng lý thuyết và công nghệ của MeetingMind theo trình tự dòng dữ liệu: bài toán trợ lý cuộc họp, nhận dạng tiếng nói, tách người nói, trích xuất thông tin và hỏi đáp bằng mô hình ngôn ngữ lớn, kiến trúc phần mềm, thu nhận nội dung họp trực tuyến, tích hợp nền tảng quản lý công việc và kiểm soát truy cập. Với mỗi nội dung, chúng tôi nêu khái niệm và lý do lựa chọn kỹ thuật cho hệ thống.

2.1. Bài toán trợ lý cuộc họp

2.1.1. Phát biểu bài toán

Đầu vào của bài toán là nội dung một cuộc họp — tín hiệu âm thanh hoặc phụ đề kèm tên người nói — cùng ngữ cảnh như dự án, người tham dự và ngày họp. Đầu ra gồm: bản ghi lời nói (transcript) chia theo lượt, có mốc thời gian và nhãn người nói; biên bản gồm tóm tắt, quyết định và vấn đề còn mở; danh sách đầu việc (action item) có mô tả, người thực hiện, thời hạn, mức ưu tiên và bằng chứng trong bản ghi. Sau khi được duyệt, đầu việc trở thành công việc chính thức và có thể đồng bộ sang công cụ quản lý bên ngoài.

Bài toán được phân rã thành chuỗi (pipeline) các khâu: nhận dạng tiếng nói, tách người nói, trích xuất thông tin, duyệt bởi con người và tích hợp. Cách này cho phép chọn công cụ tốt nhất cho từng khâu, nhưng lỗi khâu trước lan sang khâu sau — nhận dạng sai một cái tên làm sai người thực hiện — nên mỗi khâu cần lưu kết quả trung gian theo phiên bản để người dùng sửa và chạy lại.

2.1.2. Đặc thù của hội thoại cuộc họp

Lời nói trong họp là lời tự phát, chứa ngập ngừng, câu bỏ dở và tự sửa ("ngày 15… à không, ngày 18"). Nhiều người nói với lượt ngắn, có lúc nói chồng; ở phòng họp, một micro chung thu tiếng từ nhiều khoảng cách. Một đầu việc thường hình thành qua nhiều lượt và người được giao có thể chỉ được nhắc ở ngôi thứ ba: khi ai đó nói "Lan làm phần này", người nói không phải là Lan. Hệ thống còn phải phân biệt đề xuất, câu hỏi, đồng ý, quyết định, giao việc và bỏ qua chuyện ngoài lề. Với tiếng Việt, họp kỹ thuật thường xen thuật ngữ tiếng Anh và dùng mốc thời gian tương đối ("thứ Sáu", "cuối tuần").

2.1.3. Ngữ liệu và hướng nghiên cứu liên quan

Ngữ liệu AMI [6] gồm khoảng một trăm giờ họp tiếng Anh, ghi đồng thời bằng micro đeo gần miệng và micro đặt xa, có chú giải nhiều tầng (phiên âm, lượt nói, hành

vi hội thoại, tóm tắt), là chuẩn so sánh cho nhiều bài toán về cuộc họp. QMSum [33] đặt bài toán tóm tắt theo truy vấn trên nhiều lĩnh vực với cách làm hai bước "định vị rồi tóm tắt", vì thông tin cần thiết nằm rải rác trong bản ghi dài. Các ngữ liệu này đều bằng tiếng Anh; tiếng Việt chưa có ngữ liệu họp công khai tương đương, nên đề tài tự dựng bộ dữ liệu có kiểm soát để đánh giá.

2.1.4. Các giải pháp trợ lý cuộc họp hiện có

Có ba cách thu nhận nội dung chính: đưa bot vào phòng để ghi âm, đọc phụ đề do nền tảng họp sinh ra, và nhận bản ghi có sẵn qua tải tệp hoặc giao diện lập trình. Bảng 2.1 tóm tắt các giải pháp tiêu biểu.

Bảng 2.1. *So sánh các giải pháp trợ lý cuộc họp tiêu biểu*

Giải pháp	Cách thu nhận	Điểm mạnh	Hạn chế với nhóm nhỏ tại Việt Nam
Otter.ai, Fireflies.ai	Bot vào phòng họp, ghi âm	Tách người nói tốt, tóm tắt, nhiều tích hợp	Chi phí bản quyền; tiếng Việt chưa phải trọng tâm; dữ liệu xử lý ở nước ngoài
Tactiq	Tiện ích trình duyệt đọc phụ đề	Không cần bot; có sẵn tên người nói	Phụ thuộc phụ đề của nền tảng; không có âm thanh để nghe lại
Microsoft Teams Premium	Tích hợp sẵn trong Teams	Liền mạch trong hệ sinh thái Microsoft	Gắn với một nền tảng; khó tuỳ biến luồng dữ liệu
FPT.AI, Viettel AI	Giao diện lập trình chuyển giọng nói thành văn bản	Nhận dạng tiếng Việt tốt	Dừng ở bản ghi thô hoặc tóm tắt đơn giản

Bot cho âm thanh gốc nhưng cần chủ phòng cho vào; phụ đề có sẵn tên người nói nhưng phụ thuộc nền tảng; tải tệp dùng được cho họp trực tiếp nhưng không tự động. Vì vậy MeetingMind hỗ trợ cả ba đường, hội tụ về một luồng xử lý chung.

2.2. Nhận dạng tiếng nói tự động

2.2.1. Các hướng tiếp cận

Nhận dạng tiếng nói tự động (Automatic Speech Recognition, ASR) tìm chuỗi từ có xác suất lớn nhất khi biết chuỗi đặc trưng âm thanh, thường trích từ các khung khoảng 25 ms, bước dịch 10 ms. Hướng truyền thống tách bài toán theo công thức Bayes thành mô hình âm học và mô hình ngôn ngữ. Mô hình âm học dùng mô hình Markov ẩn (Hidden Markov Model, HMM) cho các âm vị phụ thuộc ngữ cảnh, xác suất phát xạ mô hình hoá bằng hỗn hợp Gauss (Gaussian Mixture Model, GMM), về sau bằng mạng nơ-ron sâu; kèm từ điển phát âm và mô hình ngôn ngữ n-gram. Kaldi là

bộ công cụ tiêu biểu; với tiếng Việt, Luong và Vu công bố bộ ngữ liệu VIVOS cùng công thức Kaldi [17], lưu ý đặc điểm đơn âm tiết và có thanh điệu của tiếng Việt.

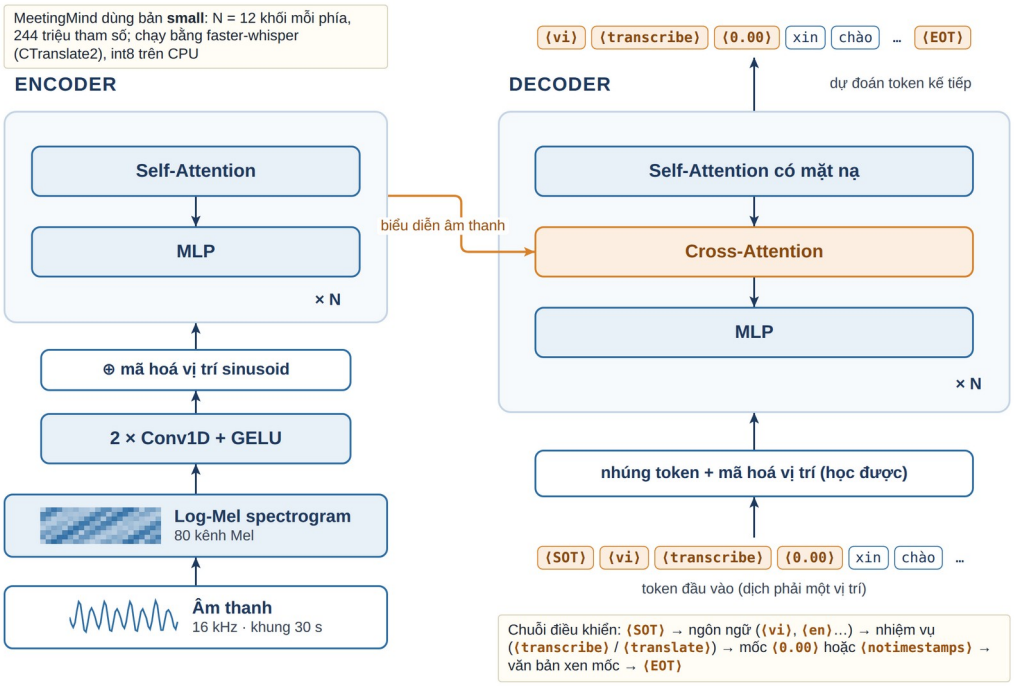
Hướng đầu-cuối (end-to-end) học trực tiếp ánh xạ âm thanh sang ký tự. Hàm mất mát phân loại theo thời gian kết nối (Connectionist Temporal Classification, CTC) thêm một ký hiệu trống, gộp nhãn lặp và cộng xác suất trên mọi cách căn chỉnh, nên không cần căn chỉnh từng khung; đổi lại, CTC giả định đầu ra các khung độc lập có điều kiện. wav2vec 2.0 [3] tiền huấn luyện tự giám sát trên âm thanh không nhãn bằng cách che một phần biểu diễn tiềm ẩn và học phân biệt biểu diễn đúng với mẫu nhiễu, rồi tinh chỉnh bằng CTC với ít dữ liệu có nhãn. Hướng thứ ba là mô hình chuỗi sang chuỗi (sequence-to-sequence) có cơ chế chú ý, ngày nay dựa trên Transformer [30]: bộ giải mã sinh từng token dựa trên các token trước nên tự học mô hình ngôn ngữ và dấu câu, nhưng giải mã tự hồi quy chậm hơn và có thể sinh nội dung không có trong âm thanh. Bảng 2.2 so sánh ba hướng này.

Bảng 2.2. *So sánh các hướng tiếp cận nhận dạng tiếng nói*

Hướng tiếp cận	Thành phần chính	Ưu điểm	Hạn chế
HMM-GMM, lai DNN-HMM (Kaldi)	Mô hình âm học, từ điển phát âm, mô hình ngôn ngữ n-gram	Cần ít dữ liệu; kiểm soát từng thành phần	Nhiều thành phần; cần chuyên môn ngôn ngữ; kém bền với miền mới
CTC, wav2vec 2.0	Bộ mã hoá tự giám sát, tinh chỉnh bằng CTC	Tận dụng âm thanh không nhãn; suy luận nhanh	Giả định độc lập giữa các khung; thường cần thêm mô hình ngôn ngữ
Chuỗi sang chuỗi (Whisper)	Bộ mã hoá và giải mã Transformer	Đa ngôn ngữ, đa nhiệm vụ; có dấu câu; bền vững	Giải mã chậm hơn; có thể ảo giác ở đoạn lặng, ồn

2.2.2. Mô hình Whisper

Whisper [23] là mô hình chuỗi sang chuỗi huấn luyện theo hướng giám sát yếu (weak supervision) trên khoảng 680 nghìn giờ âm thanh kèm văn bản thu thập từ Internet, nhiều ngôn ngữ và nhiều nhiệm vụ. Âm thanh được lấy mẫu lại ở 16 kHz, cắt thành cửa sổ 30 giây và chuyển thành phổ log-Mel. Bộ mã hoá gồm hai lớp tích chập, lớp thứ hai có bước nhảy 2, theo sau là các khối Transformer; bộ giải mã Transformer sinh văn bản tự hồi quy, có chú ý chéo (cross-attention) tới đầu ra bộ mã hoá (Hình 2.1).



Hình 2.1. *Kiến trúc mô hình Whisper với chuỗi token đặc biệt điều khiển ngôn ngữ, nhiệm vụ và mốc thời gian*

Whisper dùng chuỗi token đặc biệt ở đầu bộ giải mã để chỉ định nhiệm vụ: token bắt đầu bản ghi, token ngôn ngữ (ví dụ `<|vi|>`), token nhiệm vụ (phiên âm hay dịch sang tiếng Anh) và token cho biết có sinh mốc thời gian hay không; mốc thời gian là token rời rạc độ phân giải 20 ms. Nhờ đó một mô hình làm được phiên âm, dịch, nhận diện ngôn ngữ và phát hiện đoạn không có lời. Whisper có nhiều kích thước, từ tiny (khoảng 39 triệu tham số), base, small (khoảng 244 triệu), medium đến large (khoảng 1,55 tỷ); mô hình lớn chính xác hơn nhưng chậm hơn nhiều. Tiếng Việt chỉ chiếm phần nhỏ trong dữ liệu huấn luyện nên độ chính xác thấp hơn tiếng Anh.

2.2.3. faster-whisper, CTranslate2 và lượng tử hoá int8

faster-whisper [27] hiện thực lại Whisper trên CTranslate2, bộ máy suy luận Transformer viết bằng C++ với các tối ưu như hợp nhất phép toán, quản lý bộ nhớ và lượng tử hoá trọng số; theo nhóm phát triển, nó nhanh hơn đáng kể và tốn ít bộ nhớ hơn bản gốc với cùng độ chính xác. Thư viện tích hợp bộ phát hiện tiếng nói (Voice Activity Detection, VAD) Silero để bỏ đoạn không có lời, và tính mốc thời gian từng từ dựa trên trọng số chú ý chéo.

Lượng tử hoá int8 biểu diễn trọng số số thực 32 bit bằng số nguyên 8 bit. Với lượng tử hoá đối xứng, mỗi nhóm trọng số w (ví dụ một hàng ma trận) có hệ số tỷ lệ $s = \max|w| / 127$; giá trị lưu là $q = \text{round}(w / s)$ và khi tính toán $w \approx s \times q$. Bộ nhớ trọng số giảm khoảng bốn lần, phép nhân ma trận dùng được lệnh số nguyên của CPU, đổi

lại sai số làm tròn nhỏ. Đây là lý do MeetingMind chọn faster-whisper với mô hình small, chạy CPU ở chế độ int8.

2.2.4. Nhận diện ngôn ngữ và hiện tượng chuyển mã Việt–Anh

Whisper nhận diện ngôn ngữ bằng phân bố xác suất của token ngôn ngữ ở bước giải mã đầu tiên; với bản ghi dài, cách dùng phổ biến chỉ nhận diện trên 30 giây đầu rồi giải mã cả bài bằng ngôn ngữ đó. Giả định này không đúng với hiện tượng chuyển mã (code-switching), khi người nói chuyển qua lại giữa hai ngôn ngữ. Chuyển mã trong câu (xen "deploy", "API" vào câu tiếng Việt) thường được Whisper xử lý chấp nhận được ở chế độ tiếng Việt; chuyển mã giữa các câu thì câu tiếng Anh bị ép giải mã như tiếng Việt, cho ra chuỗi vô nghĩa hoặc bị dịch.

Cách khắc phục là nhận diện lại ngôn ngữ cho từng đoạn. Do đoạn ngắn chứa quá ít âm thanh để nhận diện tin cậy, chỉ nên đổi ngôn ngữ của đoạn khi đoạn đủ dài và xác suất nhận diện vượt ngưỡng đủ cao; còn lại giữ ngôn ngữ chính. Cơ chế phiên âm hai lượt dựa trên ý tưởng này được trình bày ở phần xây dựng hệ thống.

2.2.5. Hiện tượng ảo giác của Whisper và cách hạn chế

Bộ giải mã của Whisper về bản chất là mô hình ngôn ngữ có điều kiện. Khi đoạn âm thanh chỉ có im lặng hoặc nhiễu, bằng chứng âm học yếu, mô hình có xu hướng sinh câu có xác suất tiên nghiệm cao trong dữ liệu huấn luyện; vì dữ liệu lấy từ phụ đề video trên mạng, đó thường là "đăng ký kênh", "cảm ơn các bạn đã theo dõi" hay "thanks for watching". Việc điều kiện hoá trên văn bản đoạn trước còn có thể khuếch đại lỗi thành vòng lặp lặp lại một câu.

Bài báo Whisper [23] đề xuất cho bản ghi dài: tìm kiếm chùm (beam search), giải mã lại với nhiệt độ cao hơn khi xác suất trung bình quá thấp hoặc văn bản bị nén quá mạnh (dấu hiệu lặp), và bỏ đoạn khi xác suất "không có lời" cao. MeetingMind dùng thêm bốn biện pháp: lọc khoảng lặng bằng VAD; tắt điều kiện hoá trên văn bản trước (`condition_on_previous_text=False`); bỏ qua khoảng lặng dài khi có dấu hiệu ảo giác (`hallucination_silence_threshold` của faster-whisper); và lọc hậu kỳ các câu rác điển hình theo danh sách mẫu.

2.2.6. Độ đo đánh giá nhận dạng tiếng nói

Độ đo phổ biến nhất là tỷ lệ lỗi từ (Word Error Rate, WER). Câu nhận dạng được căn chỉnh với câu tham chiếu theo khoảng cách chỉnh sửa nhỏ nhất [16], tính bằng quy hoạch động, rồi đếm các phép sửa:

$$WER = (S + D + I) / N$$

trong đó S là số từ bị thay thế, D là số từ bị xoá (bỏ sót), I là số từ bị chèn thêm, N là số từ của câu tham chiếu. Do có phép chèn, WER có thể vượt 100%. Tỷ lệ lỗi ký tự (Character Error Rate, CER) tính tương tự ở mức ký tự. Với tiếng Việt, khoảng trắng phân tách âm tiết chứ không phải từ, nên WER thực chất là tỷ lệ lỗi âm tiết. Trước khi tính cần chuẩn hoá: chữ thường, bỏ dấu câu, đưa Unicode về dạng dựng sẵn (NFC) và thống nhất cách viết số ("18" hay "mười tám"), vì chuẩn hoá khác nhau có thể làm WER chênh lệch đáng kể. Về tốc độ, hệ số thời gian thực (Real-Time Factor, RTF) là tỷ số giữa thời gian xử lý và thời lượng âm thanh; RTF nhỏ hơn 1 nghĩa là xử lý nhanh hơn thời gian thực.

2.3. Tách người nói

2.3.1. Định nghĩa bài toán và độ đo DER

Tách người nói (speaker diarization) trả lời câu hỏi "ai nói khi nào" [21]: chia bản ghi thành các đoạn và gán mỗi đoạn một nhãn ẩn danh như "Người nói 1", "Người nói 2". Bài toán khác nhận dạng người nói (speaker identification) vốn xác định danh tính qua mẫu giọng đã đăng ký: tách người nói chỉ biết hai đoạn cùng hay khác giọng, không biết người đó là ai, nên việc gắn tên thật do người dùng xác nhận.

Độ đo chuẩn là tỷ lệ lỗi tách người nói (Diarization Error Rate, DER):

DER = (T_FA + T_MISS + T_CONF) / T_REF

trong đó T_FA là thời lượng báo nhầm có lời, T_MISS là thời lượng lời bị bỏ sót, T_CONF là thời lượng lời bị gán nhầm người, T_REF là tổng thời lượng lời trong tham chiếu. Vì nhãn ẩn danh, trước khi tính cần tìm ánh xạ một–một tối ưu giữa nhãn hệ thống và người thật. Khi ranh giới lời nói đã lấy từ khâu nhận dạng tiếng nói, có thể dùng độ đo đơn giản hơn là độ đúng gán người nói theo thời lượng dưới ánh xạ tối ưu, tương ứng phần T_CONF của DER.

2.3.2. Vector đặc trưng giọng nói

Phần lớn hệ thống tách người nói dựa trên vector đặc trưng giọng (speaker embedding): vector có số chiều cố định đại diện cho giọng của một đoạn âm thanh, sao cho hai đoạn cùng người cho vector gần nhau. i-vector là thế hệ thống kê, biểu diễn siêu vector trung bình của mô hình GMM trong một không gian biến thiên tổng số chiều thấp. x-vector [26] chuyển sang học sâu: các lớp mạng nơ-ron trễ thời gian xử lý từng khung, lớp gộp thống kê tính trung bình và độ lệch chuẩn trên cả đoạn, các lớp sau học phân loại người nói, và vector lấy từ lớp ẩn làm đặc trưng.

ERes2Net [7] dựa trên Res2Net — kiến trúc biểu diễn đặc trưng đa tỉ lệ trong mỗi khối dư — và bổ sung cơ chế hợp nhất đặc trưng cục bộ và toàn cục bằng chú ý,

giúp vector phân biệt tốt hơn cả với đoạn âm thanh ngắn. MeetingMind dùng mô hình ERes2Net của bộ 3D-Speaker ở dạng ONNX, chạy CPU qua thư viện sherpa-onnx [14]. Độ giống giữa hai vector u, v đo bằng độ tương đồng cosine:

$$\cos(u, v) = (u \cdot v) / (\|u\| \times \|v\|)$$

giá trị gần 1 khi hai đoạn rất giống nhau; khoảng cách cosine tương ứng là 1 − cos(u, v).

2.3.3. Gom cụm phân cấp theo độ tương đồng cosine

Quy trình tách người nói dạng mô-đun gồm: phát hiện đoạn có lời, cắt thành cửa sổ ngắn, tính vector giọng cho mỗi cửa sổ và gom cụm các vector, mỗi cụm là một người nói (Hình 2.2). pyannote.audio [4] cung cấp các khối mạng nơ-ron cho từng bước, trong đó có mô hình phân đoạn học sâu. Độ dài cửa sổ là một đánh đổi: cửa sổ ngắn bám sát chỗ đổi người nhưng vector nhiễu, cửa sổ dài cho vector ổn định nhưng dễ chứa hai người. Cắt cửa sổ khoảng 1,5 giây theo ranh giới từ, dựa trên mốc thời gian từ của Whisper, giúp không cắt giữa từ.



Hình 2.2. *Quy trình tách người nói dựa trên vector đặc trưng giọng và gom cụm phân cấp*

Gom cụm phân cấp từ dưới lên (Agglomerative Hierarchical Clustering, AHC) coi mỗi cửa sổ là một cụm, lặp lại việc gộp hai cụm giống nhau nhất cho tới khi độ giống lớn nhất còn lại thấp hơn ngưỡng; với liên kết trung bình (average linkage), độ giống hai cụm là trung bình độ giống mọi cặp phần tử giữa chúng. Số người nói nhờ đó được suy ra tự động, nhưng ngưỡng phụ thuộc mô hình vector và điều kiện thu âm; cửa sổ ngắn, ồn còn có thể sinh "người nói ảo", nên thường gộp các cụm có tổng thời lượng quá nhỏ vào cụm gần nhất.

2.3.4. Vấn đề nói chồng

Nói chồng (overlapping speech) rất phổ biến trong họp: chen ngang, đáp lời ngắn, hai người cùng mở lời. Với cách tiếp cận gom cụm, mỗi cửa sổ chỉ nhận một nhãn nên lời người thứ hai bị bỏ sót, góp vào T_MISS; khâu nhận dạng một kênh cũng thường chỉ chép được giọng át. Các hướng xử lý triệt để gồm mô hình phát hiện nói chồng chuyên biệt, mô hình tách người nói đầu-cuối cho phép nhiều nhãn cùng lúc, hoặc tách nguồn âm thanh trước khi nhận dạng [21].

Trong phạm vi đề tài, MeetingMind chỉ đánh dấu cửa sổ có vector giọng giống người nói thứ hai cao bất thường so với chính bản ghi là "có thể nói chồng" để người dùng nghe lại; đây là gợi ý hỗ trợ kiểm tra, không tách được lời người thứ hai.

2.4. Mô hình ngôn ngữ lớn trong trích xuất thông tin và hỏi đáp

2.4.1. Kiến trúc Transformer và mô hình ngôn ngữ lớn

Transformer [30] thay mạng hồi quy bằng cơ chế tự chú ý (self-attention): mỗi token được chiếu thành vector truy vấn Q, khoá K và giá trị V, đầu ra là tổ hợp có trọng số của các giá trị:

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \times K^T / \sqrt{d_k}) \times V$$

với d_k là số chiều vector khoá. Nhiều đầu chú ý chạy song song giúp nắm nhiều kiểu quan hệ, và cả chuỗi được xử lý song song khi huấn luyện. Mô hình ngôn ngữ lớn hiện nay chủ yếu chỉ dùng phần giải mã, huấn luyện dự đoán token tiếp theo trên kho văn bản rất lớn. GPT-3 [5] với 175 tỷ tham số cho thấy mô hình đủ lớn có thể làm nhiệm vụ mới chỉ từ mô tả và vài ví dụ trong ngữ cảnh.

DeepSeek-V3 [8] dùng kiến trúc hỗn hợp chuyên gia (Mixture-of-Experts, MoE) với 671 tỷ tham số nhưng mỗi token chỉ kích hoạt khoảng 37 tỷ, giảm mạnh chi phí suy luận. Họ GLM [28] là một dòng mô hình song ngữ trọng số mở khác. MeetingMind sử dụng các thế hệ kế tiếp của hai họ mô hình này.

2.4.2. Mô hình suy luận

Wei và cộng sự [32] chỉ ra rằng yêu cầu mô hình viết các bước lập luận trung gian (chain-of-thought) trước khi trả lời cải thiện rõ khả năng giải bài toán nhiều bước ở mô hình đủ lớn. Các mô hình suy luận (reasoning model) được huấn luyện để tự sinh chuỗi suy nghĩ dài trước câu trả lời, thường được giao diện lập trình trả tách riêng khỏi nội dung chính. Với trích xuất biên bản, suy luận hữu ích khi phải theo dõi cả cuộc hội thoại: thời hạn bị đính chính, đầu việc bị chuyển người, đề xuất cuối cùng không được chấp nhận. Đổi lại, mô hình suy luận tốn nhiều token và có độ trễ cao hơn hẳn; vì phân

tích chạy nền sau phiên âm nên độ trễ này chấp nhận được, nhưng giới hạn token phải đủ lớn cho cả phần suy nghĩ lẫn JSON đầu ra.

2.4.3. Thiết kế prompt và đầu ra JSON có cấu trúc

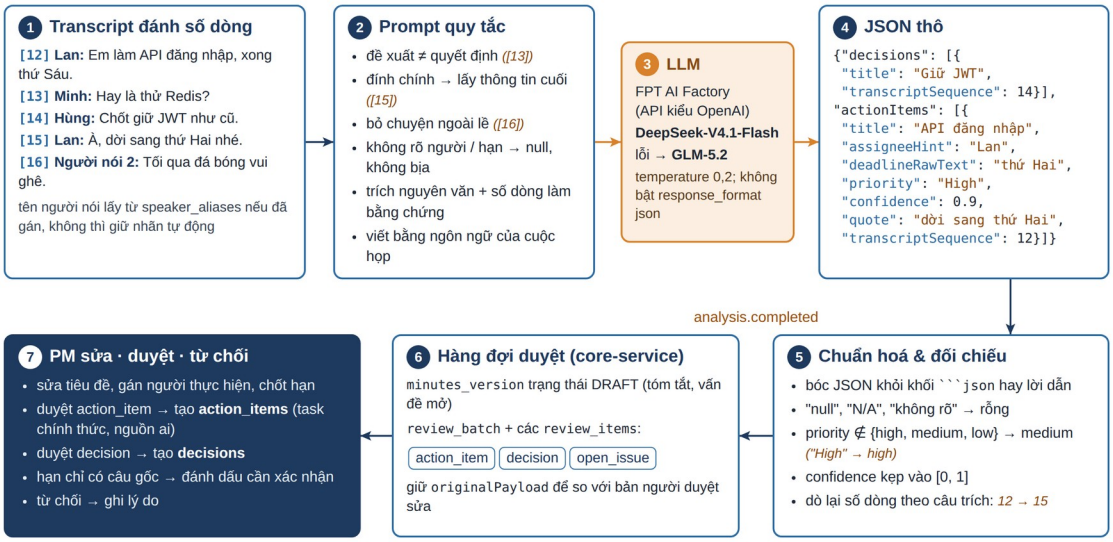
Prompt cho bài toán trích xuất gồm thông điệp hệ thống xác định vai trò (thư ký cuộc họp), các quy tắc nghiệp vụ, lược đồ đầu ra và bản ghi đã đánh số dòng theo dạng "[số thứ tự] Người nói: lời nói". Quy tắc phải nêu rõ những phân biệt con người coi là hiển nhiên: đề xuất chưa ai đồng ý không phải quyết định; có đính chính thì lấy thông tin sau cùng; bỏ chuyện ngoài lề; thiếu người thực hiện hay thời hạn thì để trống. Thời hạn được giữ nguyên văn ("thứ Sáu", "18/11"), việc quy đổi thành ngày cụ thể là bước riêng.

Một số dịch vụ hỗ trợ chế độ ép JSON hoặc giải mã ràng buộc theo lược đồ, nhưng không phải lúc nào cũng tương thích với mô hình suy luận: nhóm ghi nhận DeepSeek-V4.1-Flash trả nội dung rỗng khi bật chế độ JSON. Vì vậy cần lớp hậu xử lý vững: bóc JSON khỏi khối mã Markdown hay lời dẫn thừa, phát hiện câu trả lời bị cắt do hết giới hạn token, ép giá trị về miền hợp lệ (mức ưu tiên thuộc {high, medium, low}, độ tin cậy trong [0, 1]) và quy các chuỗi "null", "N/A" về rỗng. Nhiệt độ lấy mẫu thấp giúp kết quả ổn định hơn giữa các lần chạy.

2.4.4. Ảo giác của LLM, đối chiếu bằng chứng và con người trong vòng lặp

Ji và cộng sự [13] phân biệt ảo giác nội tại (mâu thuẫn với nguồn) và ảo giác ngoại tại (không kiểm chứng được từ nguồn). Trong trích xuất biên bản, ảo giác biểu hiện thành người thực hiện hoặc thời hạn bịa ra, đề xuất bị ghi thành quyết định, thông tin cũ được giữ dù đã đính chính, hay trích dẫn sai dòng — nguy hiểm vì đầu ra trông hợp lý.

Biện pháp thứ nhất là buộc mỗi quyết định, đầu việc kèm câu trích nguyên văn và số dòng nguồn, rồi đối chiếu tự động sau khi chuẩn hoá văn bản; nếu câu trích không nằm trong dòng được chỉ ra thì dò lại dòng chứa nó. Biện pháp thứ hai là đưa con người vào vòng lặp (human-in-the-loop): kết quả của AI chỉ là đề xuất trong hàng đợi duyệt, chỉ đề xuất được người quản lý duyệt mới thành công việc chính thức (Hình 2.3). Độ tin cậy do mô hình tự khai không phải xác suất đã hiệu chỉnh, nên chỉ dùng để gợi ý mức chú ý.



Hình 2.3. *Trích xuất thông tin có cấu trúc bằng LLM, đối chiếu bằng chứng và duyệt bởi con người*

2.4.5. Sinh văn bản tăng cường truy xuất

Mô hình ngôn ngữ chỉ biết những gì có trong dữ liệu huấn luyện và trong ngữ cảnh được đưa vào; nó không biết nội dung các cuộc họp nội bộ của một nhóm, và nếu bị hỏi về chúng thì dễ sinh câu trả lời trôi chảy nhưng không có căn cứ — dạng ảo giác ngoại tại [13]. Sinh văn bản tăng cường truy xuất (Retrieval-Augmented Generation, RAG) tách việc trả lời thành hai bước: trước hết truy xuất từ kho tài liệu những đoạn liên quan tới câu hỏi, sau đó đưa các đoạn này vào prompt và yêu cầu mô hình chỉ trả lời dựa trên chúng. Kho tri thức được cập nhật bằng cách lập chỉ mục lại, không phải huấn luyện lại mô hình.

Bước truy xuất thường dùng biểu diễn vector dày đặc: một mô hình nhúng (embedding model), thường là bộ mã hoá Transformer [30], ánh xạ mỗi đoạn văn và câu hỏi thành một vector có số chiều cố định sao cho văn bản gần nghĩa có độ tương đồng cosine cao. Tài liệu dài được chia thành đoạn (chunk) trước khi nhúng; kích thước đoạn là thế cân bằng giữa đủ ngữ cảnh cho một ý và đủ nhỏ để trích dẫn sát nội dung, vừa với cửa sổ ngữ cảnh của mô hình. Với hội thoại, ranh giới tự nhiên là lượt nói: cắt ngang một lượt nói làm mất người nói và mốc thời gian cần cho trích dẫn.

Tìm chính xác k láng giềng gần nhất đòi hỏi so câu hỏi với mọi vector, nên các hệ thống dùng chỉ mục tìm kiếm gần đúng. HNSW (Hierarchical Navigable Small World) xây một đồ thị nhiều tầng: tầng trên thưa để nhảy xa, tầng dưới dày để tinh chỉnh; truy vấn đi tham lam từ tầng trên xuống, đạt độ phủ cao với số phép so sánh tăng chậm theo kích thước kho. Phần mở rộng pgvector cho phép lưu vector và chỉ mục HNSW ngay trong PostgreSQL, nhờ đó điều kiện lọc theo thuộc tính và sắp xếp theo khoảng cách nằm trong cùng một câu SQL. Khi kết hợp lọc với chỉ mục gần đúng

cần lưu ý: nếu bộ lọc được áp sau khi duyệt đồ thị, số kết quả có thể bị hụt; pgvector từ phiên bản 0.8 có chế độ quét lặp (iterative scan) duyệt tiếp cho tới khi đủ số dòng thoả điều kiện.

Độ tương đồng vector chủ yếu đo mức "cùng chủ đề", vì câu hỏi và đoạn văn được mã hoá độc lập (bi-encoder). Bộ xếp hạng lại (reranker) dạng cross-encoder đọc đồng thời cặp câu hỏi – đoạn văn nên đánh giá được đoạn đó có thật sự trả lời câu hỏi hay không; cách này chính xác hơn nhưng tốn kém hơn, nên chỉ áp lên vài chục ứng viên đầu của bước tìm vector rồi giữ lại vài đoạn tốt nhất.

Ở bước sinh, để hạn chế ảo giác, prompt buộc mô hình chỉ dùng các nguồn được đánh số, gắn số nguồn [n] sau mỗi ý và trả lời rõ "chưa đủ thông tin" khi nguồn không chứa đáp án; trích dẫn cho phép người đọc kiểm chứng câu trả lời thay vì phải tin mô hình [13]. Trong hệ thống nhiều người dùng còn một yêu cầu an toàn: kiểm soát truy cập phải được áp *trước* bước truy xuất. Nếu chỉ lọc sau khi đã lấy đoạn văn hoặc sau khi mô hình trả lời, nội dung người hỏi không được xem đã lọt vào ngữ cảnh và có thể được diễn đạt lại trong câu trả lời — sai sót không thể gỡ khỏi văn bản đã sinh. Vì vậy, điều kiện phân quyền phải là một phần của chính truy vấn tìm kiếm.

2.4.6. Dịch vụ suy luận và chủ quyền dữ liệu

Có ba cách đưa LLM vào hệ thống: tự vận hành mô hình trọng số mở trên GPU riêng, chủ động nhưng tốn kém; gọi dịch vụ nước ngoài, dễ dùng nhưng dữ liệu hội thoại rời khỏi phạm vi kiểm soát; hoặc dùng dịch vụ suy luận trong nước. FPT AI Factory [9] cung cấp suy luận phi máy chủ với giao diện tương thích OpenAI, tính phí theo token. Cần phân biệt mô hình do FPT tự vận hành, dữ liệu xử lý trên hạ tầng FPT, với mô hình bán lại (resell) từ Azure, Google, Alibaba…, khi đó yêu cầu được chuyển ra ngoài. Vì nội dung họp có thể nhạy cảm, MeetingMind chỉ dùng mô hình do FPT tự vận hành: DeepSeek-V4.1-Flash làm mô hình chính, GLM-5.2 làm dự phòng; đổi mô hình chỉ cần sửa cấu hình.

2.5. Kiến trúc microservices và hướng sự kiện

2.5.1. Microservices và cổng API

Kiến trúc microservices tổ chức hệ thống thành các service nhỏ, triển khai độc lập, mô hình hoá quanh miền nghiệp vụ và sở hữu dữ liệu riêng [20]; mỗi phần dùng được công nghệ phù hợp — dịch vụ AI viết bằng Python, nghiệp vụ viết bằng NestJS [19] — đổi lại là độ phức tạp của hệ phân tán. Nguyên tắc cơ sở dữ liệu riêng cho mỗi service [24] đòi hỏi mỗi bảng chỉ có một service được ghi; service khác lấy dữ liệu qua API hoặc sự kiện. Cổng API (API Gateway) là điểm vào duy nhất, xác thực token,

định tuyến và che giấu cấu trúc nội bộ; trong MeetingMind, cổng chỉ kiểm tra token, còn quyết định quyền nghiệp vụ thuộc về service lõi.

2.5.2. Giao tiếp bất đồng bộ qua RabbitMQ

Phiên âm một giờ họp có thể mất nhiều phút, nên không thể giữ yêu cầu HTTP chờ kết quả; với hàng đợi thông điệp, bên gửi đặt việc vào hàng đợi rồi trả lời ngay, bên xử lý nhận việc theo năng lực của mình. RabbitMQ [22] là bộ môi giới thông điệp theo giao thức AMQP 0-9-1: bên gửi đẩy thông điệp vào exchange, exchange định tuyến tới hàng đợi theo khoá định tuyến (routing key) và quy tắc liên kết (binding); bên nhận gửi xác nhận (acknowledgement) sau khi xử lý xong. Hàng đợi và thông điệp lưu bền (durable) giúp không mất việc khi bộ môi giới khởi động lại; tham số prefetch giới hạn số thông điệp một bên nhận giữ cùng lúc. Thông điệp chỉ nên mang tham chiếu (mã cuộc họp, số phiên bản, khoá đối tượng) thay vì cả tệp âm thanh; tệp lớn đặt trong kho lưu trữ đối tượng tương thích S3 như MinIO.

2.5.3. Ngữ nghĩa giao nhận và tính lũy đẳng

Hệ thống thông điệp có ba mức bảo đảm: nhiều nhất một lần (at-most-once) có thể mất thông điệp; ít nhất một lần (at-least-once) không mất nhưng có thể trùng; đúng một lần (exactly-once) rất khó đạt trong hệ phân tán. Với xác nhận thủ công, nếu bên nhận gặp sự cố sau khi xử lý nhưng trước khi xác nhận, RabbitMQ giao lại thông điệp. Kleppmann [15] chỉ ra rằng hiệu quả "đúng một lần" đạt được bằng giao nhận ít nhất một lần kết hợp xử lý lũy đẳng (idempotent): xử lý nhiều lần cho cùng kết quả như một lần.

Mẫu bên nhận lũy đẳng [24] ghi mã định danh mỗi thông điệp đã xử lý vào một bảng (trong MeetingMind là `processed_messages`) trong cùng giao dịch với thay đổi nghiệp vụ; gặp lại mã đã có thì bỏ qua. Ở phía yêu cầu, mỗi công việc xử lý mang một khoá lũy đẳng (idempotency key) duy nhất để yêu cầu gửi lại không tạo việc trùng.

2.5.4. Mẫu transactional outbox

Khi một service vừa cập nhật cơ sở dữ liệu vừa phát sự kiện, hai thao tác không nằm trong cùng giao dịch: một bên thành công còn bên kia thất bại sẽ làm hệ thống mất nhất quán — vấn đề ghi kép (dual write). Mẫu transactional outbox [24] ghi sự kiện vào bảng outbox trong cùng giao dịch cục bộ với thay đổi nghiệp vụ; một tiến trình chuyển phát riêng đọc bảng outbox, đẩy sự kiện lên bộ môi giới và đánh dấu đã gửi. Vì sự kiện có thể bị gửi lại khi tiến trình chuyển phát lỗi giữa chừng, mẫu này đi kèm bên nhận lũy đẳng; kết hợp hai mẫu, mỗi yêu cầu phiên âm hay phân tích có hiệu quả đúng một lần, kể cả khi service khởi động lại.

2.6. Thu nhận nội dung họp trực tuyến

2.6.1. WebRTC và xử lý âm thanh trên trình duyệt

WebRTC [31] là chuẩn truyền âm thanh, hình ảnh thời gian thực giữa các trình duyệt. Đối tượng `RTCPeerConnection` thiết lập kết nối qua cơ chế ICE, truyền dữ liệu đã mã hoá và phát sự kiện `track` mỗi khi nhận một luồng từ phía bên kia. Dịch vụ họp nhiều người thường dùng máy chủ trung gian chuyển tiếp luồng, nhưng ở phía trình duyệt, âm thanh của người khác vẫn đến dưới dạng các luồng từ xa của `RTCPeerConnection`.

Web Audio API với `AudioContext` cho phép trộn nhiều nguồn âm thanh, còn MediaRecorder ghi một luồng thành tệp nén (ví dụ WebM mã hoá Opus) theo từng khúc. Nhờ đó bot có thể ghi âm ngay trong trình duyệt của nó: bọc hàm khởi tạo `RTCPeerConnection` để bắt mọi luồng âm thanh từ xa, trộn lại rồi ghi — tức ghi đúng những gì một người tham dự nghe được.

2.6.2. Tự động hoá trình duyệt với Playwright và Xvfb

Playwright [18] là thư viện điều khiển Chromium, Firefox và WebKit bằng mã nguồn, cho phép chèn mã chạy trước mọi mã của trang (init script), cho mã trên trang gọi ngược về chương trình điều khiển và dùng hồ sơ trình duyệt lưu bền để giữ phiên đăng nhập. Chế độ không giao diện (headless) dễ bị nền tảng nhận ra, nên bot họp thường chạy trình duyệt có giao diện trên màn hình ảo: Xvfb (X virtual framebuffer) là máy chủ đồ hoạ X vẽ vào bộ nhớ, cho phép chạy trình duyệt đầy đủ trong container; PulseAudio cung cấp thiết bị âm thanh ảo.

Các nền tảng họp có cơ chế phát hiện trình duyệt tự động; thực tế Google Meet chặn khách ẩn danh dùng trình duyệt tự động, nên bot cần tài khoản Google đã đăng nhập. Về đạo đức và tuân thủ, bot không vượt cơ chế kiểm soát của nền tảng: hiện đúng tên, phải được chủ phòng cho vào và dừng khi bị từ chối.

2.6.3. Tiện ích mở rộng Chrome Manifest V3

Manifest V3 [10] là thế hệ nền tảng tiện ích hiện hành của Chrome. Trang nền chạy liên tục được thay bằng service worker hướng sự kiện, có thể bị dừng khi rảnh, nên trạng thái phải lưu vào `chrome.storage`; mã thực thi phải đóng gói sẵn, không tải từ xa; quyền truy cập trang được khai báo tường minh. Content script được tiêm vào các trang khớp mẫu địa chỉ và chạy trong "thế giới cô lập" (isolated world): dùng chung DOM với trang nhưng không chung biến JavaScript, trao đổi với service worker bằng truyền thông điệp.

Đọc phụ đề Google Meet là bài toán quan sát DOM. Tên lớp CSS của Meet bị làm rối và đổi định kỳ, nên cần dựa vào cấu trúc DOM và thuộc tính trợ năng ổn định hơn như `role`, `aria-label`; vùng phụ đề còn bị dựng lại sau khoảng im lặng, nên quét định kỳ bền hơn gắn bộ quan sát cố định. Phụ đề là kết quả nhận dạng của chính Meet, không kèm âm thanh: đường thu nhận này nhẹ, có sẵn tên người nói, nhưng chất lượng phụ thuộc nền tảng.

2.7. Tích hợp nền tảng quản lý công việc

Mục này trình bày cơ sở lý thuyết và kết quả khảo sát cho việc đồng bộ ra nền tảng ngoài. Ở thời điểm báo cáo, MeetingMind đã hiện thực đồng bộ một chiều công việc đã duyệt sang Trello và thông báo công việc mới qua Telegram (trình bày ở chương Xây dựng hệ thống); đồng bộ Jira và cơ chế kết nối OAuth theo từng workspace vẫn ở mức thiết kế.

2.7.1. OAuth 2.0 và Webhook

OAuth 2.0 [11] là khung uỷ quyền cho phép ứng dụng truy cập tài nguyên của người dùng trên dịch vụ khác mà không cần mật khẩu, với bốn vai trò: chủ sở hữu tài nguyên, ứng dụng khách, máy chủ uỷ quyền và máy chủ tài nguyên. Trong luồng mã uỷ quyền (authorization code), người dùng đồng ý với một phạm vi quyền (scope) tại máy chủ uỷ quyền; ứng dụng nhận mã rồi đổi lấy access token có thời hạn ngắn và refresh token để gia hạn. Token phải lưu bảo mật phía máy chủ, không trả về giao diện.

Webhook là cơ chế đẩy ngược: thay vì ứng dụng hỏi định kỳ (polling), nền tảng gọi một địa chỉ HTTP đã đăng ký mỗi khi có sự kiện; bên nhận cần xác minh nguồn gửi và xử lý lũy đẳng vì nền tảng có thể gửi lại.

2.7.2. Jira, Trello và Telegram

Jira Cloud cung cấp REST API phiên bản 3 [1], xác thực bằng OAuth 2.0 hoặc API token. Đối tượng trung tâm là issue thuộc một project, có tiêu đề, mô tả theo định dạng tài liệu Atlassian (Atlassian Document Format), người được giao định danh bằng `accountId`, hạn chót và mức ưu tiên; trạng thái đổi qua các bước chuyển (transition). Trello [2] tổ chức dữ liệu theo bảng (board), danh sách (list) và thẻ (card); tạo thẻ cần chỉ định danh sách đích, kèm mô tả, hạn và thành viên; quyền truy cập cấp bằng khoá API kèm token do người dùng uỷ quyền. Telegram Bot API [29] cho phép bot gửi tin qua HTTPS bằng phương thức `sendMessage`, nhận cập nhật bằng `getUpdates` hoặc webhook; bot không thể nhắn trước cho người chưa mở trò chuyện với nó, nên người

dùng phải khởi động bot trước để hệ thống lưu định danh cuộc trò chuyện. Bảng 2.3 tóm tắt các đặc điểm này.

Bảng 2.3. *Đặc điểm các nền tảng tích hợp được khảo sát*

Nền tảng	Cơ chế cấp quyền	Đối tượng chính	Thao tác dự kiến	Nhận thay đổi
Jira Cloud	OAuth 2.0 hoặc API token	Project, issue, transition	Tạo issue từ công việc đã duyệt	Webhook
Trello	Khoá API kèm token người dùng	Board, list, card	Tạo thẻ trong danh sách đích	Webhook
Telegram	Token của bot	Chat, message	Thông báo công việc mới, sắp đến hạn	getUpdates hoặc webhook

2.8. Kiểm soát truy cập

2.8.1. Mô hình RBAC

Kiểm soát truy cập dựa trên vai trò (Role-Based Access Control, RBAC) [25] gán quyền cho vai trò và vai trò cho người dùng. Họ mô hình RBAC96 gồm RBAC0 là lõi (người dùng, vai trò, quyền, phiên làm việc), RBAC1 thêm phân cấp vai trò, RBAC2 thêm ràng buộc như phân tách nhiệm vụ, RBAC3 kết hợp cả hai. RBAC dễ quản trị và kiểm toán, nhưng vai trò toàn cục không diễn đạt được ngữ cảnh: muốn nói "PM của dự án A nhưng chỉ là thành viên dự án B" phải sinh vai trò riêng cho từng dự án, dẫn tới bùng nổ số vai trò.

2.8.2. Mô hình ABAC

Kiểm soát truy cập dựa trên thuộc tính (Attribute-Based Access Control, ABAC), theo NIST SP 800-162 [12], quyết định cho phép hay từ chối dựa trên thuộc tính của chủ thể, đối tượng, môi trường và tập chính sách biểu diễn bằng các thuộc tính đó. ABAC diễn đạt được điều kiện như "người tạo cuộc họp", "người được giao việc", "tài nguyên đang ở trạng thái nháp", nhưng chính sách phức tạp khó kiểm toán và khó trả lời câu hỏi "ai đang có quyền với tài nguyên này".

2.8.3. Phân quyền theo vai trò kết hợp phạm vi tài nguyên

Giải pháp dung hoà là gán vai trò trong phạm vi một tài nguyên, kết hợp thừa kế theo cây tài nguyên và điều kiện thuộc tính bổ sung: người dùng có vai trò ở từng tầng — hệ thống, workspace, dự án, cuộc họp độc lập; cuộc họp thuộc dự án kế thừa quyền của dự án; một số quyền kèm điều kiện như "chỉ với cuộc họp mình tạo". Cách làm này giữ tính dễ hiểu của RBAC, tránh bùng nổ vai trò, chỉ dùng thuộc tính ở nơi cần.

Đi kèm là nguyên tắc đặc quyền tối thiểu (least privilege) — mặc định từ chối — và tách quyền quản trị khỏi quyền đọc nội dung: quản trị viên hệ thống không mặc nhiên đọc được cuộc họp riêng tư. Bảng 2.4 so sánh ba cách tiếp cận.

Bảng 2.4. *So sánh các mô hình kiểm soát truy cập*

Tiêu chí	RBAC	ABAC	Vai trò theo phạm vi tài nguyên
Căn cứ quyết định	Vai trò toàn cục	Thuộc tính và chính sách	Vai trò tại tài nguyên hoặc tài nguyên cha, cộng điều kiện bổ sung
Diễn đạt ngữ cảnh	Kém, dễ bùng nổ vai trò	Rất tốt	Tốt với dữ liệu phân cấp
Độ phức tạp quản trị	Thấp	Cao	Trung bình
Kiểm toán "ai có quyền gì"	Dễ	Khó	Tương đối dễ
Phù hợp	Hệ thống ít ngữ cảnh	Chính sách động, nhiều điều kiện	Ứng dụng nhiều tổ chức, dữ liệu phân cấp

2.9. Kết luận chương

Chương này đã trình bày cơ sở cho từng khâu của MeetingMind. faster-whisper với lượng tử hoá int8 cho phép nhận dạng tiếng nói trên CPU, kèm hai vấn đề cần xử lý riêng là chuyển mã Việt–Anh và ảo giác ở đoạn lặng, ồn. Vector giọng ERes2Net kết hợp gom cụm phân cấp tự suy ra số người nói, còn nói chồng vẫn là giới hạn của cách tiếp cận một kênh. LLM nắm được các tình huống khó của hội thoại nhưng cần quy tắc rõ, đối chiếu bằng chứng và con người duyệt; khi dùng để hỏi đáp, mô hình phải được neo vào các đoạn nguồn truy xuất trong phạm vi quyền của người hỏi và bắt buộc trích dẫn. Microservices hướng sự kiện với outbox và xử lý lũy đẳng phù hợp cho tác vụ AI chạy lâu; phân quyền theo vai trò kết hợp phạm vi tài nguyên dung hoà RBAC và ABAC. Đây là cơ sở cho phân tích và thiết kế hệ thống ở chương tiếp theo.

CHƯƠNG 3. KẾT QUẢ NỘI DUNG VÀ ĐÁNH GIÁ

3.1. Các lựa chọn công nghệ rút ra từ tổng quan

Từ kết quả tổng quan, nhóm xác định công nghệ chủ đạo cho từng thành phần của hệ thống như trong Bảng 3.1. Nguyên tắc chung là ưu tiên mô hình mã nguồn mở chạy được trên CPU cho các bước xử lý âm thanh (để dữ liệu ghi âm không phải rời khỏi máy chủ của tổ chức), và đặt bộ phân tích LLM sau một giao diện chung để có thể đổi nhà cung cấp mà không ảnh hưởng luồng nghiệp vụ.

Bảng 3.1. *Công nghệ chủ đạo được xác định sau bước tổng quan*

Thành phần	Công nghệ lựa chọn	Lý do chính
Nhận dạng tiếng nói	Whisper (bản faster-whisper, model small, int8 trên CPU)	Đa ngôn ngữ, tự nhận diện ngôn ngữ, chạy được trên CPU; có thể nâng lên medium/large khi có GPU
Tách người nói	Vector đặc trưng giọng ERes2Net (sherpa-onnx) + gom cụm phân cấp	Không cần token Hugging Face, chạy ONNX trên CPU, không phụ thuộc số người nói cho trước
Tóm tắt, trích xuất công việc	LLM qua API kiểu OpenAI; giai đoạn đầu dùng bộ giả lập, chốt mô hình bằng thực nghiệm ở bước kiểm thử	Tách biệt luồng nghiệp vụ với nhà cung cấp; chọn mô hình dựa trên số liệu thay vì cảm tính
Thu nhận nội dung	Ba đường: tải file ghi âm, bot vào Google Meet, extension đọc phụ đề	Mỗi đường bù cho hạn chế của đường kia (xem phần tổng quan)
Giao tiếp giữa thành phần	RabbitMQ + mẫu transactional outbox + idempotency	Xử lý AI kéo dài vài phút, cần bất đồng bộ và chống xử lý lặp
Nền tảng quản lý công việc mục tiêu	Jira Cloud và Trello (REST API, OAuth 2.0/API key, Webhook)	Phổ biến trong các nhóm phát triển phần mềm tại Việt Nam
Kênh cảnh báo	Telegram Bot API	Đơn giản, miễn phí, phổ biến
Kiểm soát truy cập	Vai trò kết hợp phạm vi tài nguyên (workspace, dự án, cuộc họp)	RBAC thuần không đủ để tách quyền theo dự án và cuộc họp

Đề cương ban đầu dự kiến dùng các LLM như GPT hoặc Gemini. Khi triển khai thực tế, nhóm chọn dịch vụ suy luận của FPT AI Factory (các mô hình do FPT tự vận hành) vì hai lý do: dữ liệu cuộc họp không phải gửi ra máy chủ nước ngoài, và chi phí cho mỗi cuộc họp rất thấp. Mô hình cụ thể (DeepSeek-V4.1-Flash làm mô hình chính, GLM-5.2 làm dự phòng) được chốt bằng thực nghiệm so sánh chín mô hình ở bước kiểm thử, trình bày trong báo cáo Nội dung 5.

3.2. Đối chiếu với kết quả cần đạt

Bảng 3.2. *Đối chiếu kết quả Nội dung 1 với đề cương*

Kết quả cần đạt theo đề cương	Kết quả thực hiện	Đánh giá
Bản tổng quan tài liệu	Tổng quan về STT, tách người nói, LLM, kiến trúc hướng sự kiện, thu nhận họp trực tuyến, tích hợp và kiểm soát truy cập (chương 2 của báo cáo này)	Đạt
Xác định mô hình AI chủ đạo	Whisper (faster-whisper) cho STT; ERes2Net + gom cụm cho tách người nói; LLM qua giao diện chung, mô hình cụ thể chốt bằng thực nghiệm	Đạt
Xác định nền tảng quản lý mục tiêu (Jira, Trello)	Đã khảo sát API, cơ chế xác thực và Webhook của Jira, Trello, Telegram; chọn làm đích đồng bộ	Đạt

Bước tổng quan đã hoàn thành đúng hạn và đủ các kết quả cần đạt. Các lựa chọn công nghệ ở bước này được giữ nguyên trong suốt quá trình xây dựng, trừ phần LLM được cụ thể hoá thêm bằng thực nghiệm ở cuối đề tài. Kết quả khảo sát API của Trello và Telegram (khoá API kèm token người dùng, danh sách và thẻ của Trello; phương thức `sendMessage`, `getUpdates` của Telegram) được dùng trực tiếp khi hiện thực mô-đun đồng bộ ở cuối đề tài (xem báo cáo Nội dung 4); với Jira, khảo sát mới dừng ở mức lý thuyết và thiết kế.

TÀI LIỆU THAM KHẢO

[1] Atlassian (2026), *Jira Cloud platform REST API v3*, https://developer.atlassian.com/cloud/jira/platform/rest/v3/, truy cập ngày 04/11/2026.

[2] Atlassian (2026), *Trello REST API*, https://developer.atlassian.com/cloud/trello/rest/, truy cập ngày 04/11/2026.

[3] Baevski A., Zhou H., Mohamed A., Auli M. (2020), "wav2vec 2.0: A framework for self-supervised learning of speech representations", *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, pp. 12449–12460.

[4] Bredin H., Yin R., Coria J. M., Gelly G., Korshunov P., Lavechin M., Fustes D., Titeux H., Bouaziz W., Gill M.-P. (2020), "pyannote.audio: neural building blocks for speaker diarization", *Proceedings of IEEE ICASSP 2020*, pp. 7124–7128.

[5] Brown T. B., Mann B., Ryder N., Subbiah M., Kaplan J., et al. (2020), "Language models are few-shot learners", *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, pp. 1877–1901.

[6] Carletta J., Ashby S., Bourban S., Flynn M., Guillemot M., Hain T., et al. (2005), "The AMI meeting corpus: A pre-announcement", *Machine Learning for Multimodal Interaction (MLMI 2005)*, LNCS 3869, Springer, pp. 28–39.

[7] Chen Y., Zheng S., Wang H., Cheng L., Chen Q., Qi J. (2023), "An enhanced Res2Net with local and global feature fusion for speaker verification", *Proceedings of Interspeech 2023*.

[8] DeepSeek-AI (2024), *DeepSeek-V3 Technical Report*, arXiv:2412.19437.

[9] FPT Smart Cloud (2026), *FPT AI Factory – Serverless Inference documentation*, https://docs.fptcloud.com/docs/category/fpt-ai-factory/, truy cập ngày 04/11/2026.

[10] Google (2026), *Chrome Extensions – Manifest V3*, https://developer.chrome.com/docs/extensions, truy cập ngày 04/11/2026.

[11] Hardt D. (Ed.) (2012), *The OAuth 2.0 Authorization Framework*, RFC 6749, Internet Engineering Task Force (IETF).

[12] Hu V. C., Ferraiolo D., Kuhn R., Schnitzer A., Sandlin K., Miller R., Scarfone K. (2014), *Guide to Attribute Based Access Control (ABAC) Definition and Considerations*, NIST Special Publication 800-162.

[13] Ji Z., Lee N., Frieske R., Yu T., Su D., Xu Y., Ishii E., Bang Y. J., Madotto A., Fung P. (2023), "Survey of hallucination in natural language generation", *ACM Computing Surveys*, 55(12), pp. 1–38.

[14] k2-fsa (2026), *sherpa-onnx: Speech-to-text, text-to-speech, speaker diarization using next-gen Kaldi with onnxruntime*, https://github.com/k2-fsa/sherpa-onnx, truy cập ngày 04/11/2026.

[15] Kleppmann M. (2017), *Designing Data-Intensive Applications*, O'Reilly Media.

[16] Levenshtein V. I. (1966), "Binary codes capable of correcting deletions, insertions, and reversals", *Soviet Physics Doklady*, 10(8), pp. 707–710.

[17] Luong H.-T., Vu H.-Q. (2016), "A non-expert Kaldi recipe for Vietnamese speech recognition system", *Proceedings of the Third International Workshop on Worldwide Language Service Infrastructure and Second Workshop on Open Infrastructures and Analysis Frameworks for Human Language Technologies (WLSI/OIAF4HLT 2016)*, pp. 51–55.

[18] Microsoft (2026), *Playwright documentation*, https://playwright.dev, truy cập ngày 04/11/2026.

[19] Myśliwiec K. (2026), *NestJS documentation*, https://docs.nestjs.com, truy cập ngày 04/11/2026.

[20] Newman S. (2021), *Building Microservices: Designing Fine-Grained Systems*, 2nd ed., O'Reilly Media.

[21] Park T. J., Kanda N., Dimitriadis D., Han K. J., Watanabe S., Narayanan S. (2022), "A review of speaker diarization: Recent advances with deep learning", *Computer Speech & Language*, 72, 101317.

[22] RabbitMQ (2026), *RabbitMQ documentation: Reliability guide, consumer acknowledgements and publisher confirms*, https://www.rabbitmq.com/docs, truy cập ngày 04/11/2026.

[23] Radford A., Kim J. W., Xu T., Brockman G., McLeavey C., Sutskever I. (2023), "Robust speech recognition via large-scale weak supervision", *Proceedings of the 40th International Conference on Machine Learning (ICML 2023)*, PMLR 202, pp. 28492–28518.

[24] Richardson C. (2018), *Microservices Patterns: With Examples in Java*, Manning Publications.

[25] Sandhu R. S., Coyne E. J., Feinstein H. L., Youman C. E. (1996), "Role-based access control models", *IEEE Computer*, 29(2), pp. 38–47.

[26] Snyder D., Garcia-Romero D., Sell G., Povey D., Khudanpur S. (2018), "X-vectors: Robust DNN embeddings for speaker recognition", *Proceedings of IEEE ICASSP 2018*, pp. 5329–5333.

[27] SYSTRAN (2026), *faster-whisper: Faster Whisper transcription with CTranslate2*, https://github.com/SYSTRAN/faster-whisper, truy cập ngày 04/11/2026.

[28] Team GLM (2024), *ChatGLM: A family of large language models from GLM-130B to GLM-4 All Tools*, arXiv:2406.12793.

[29] Telegram (2026), *Telegram Bot API*, https://core.telegram.org/bots/api, truy cập ngày 04/11/2026.

[30] Vaswani A., Shazeer N., Parmar N., Uszkoreit J., Jones L., Gomez A. N., Kaiser Ł., Polosukhin I. (2017), “Attention is all you need”, *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, pp. 5998–6008.

[31] W3C (2021), *WebRTC 1.0: Real-Time Communication Between Browsers*, W3C Recommendation, https://www.w3.org/TR/webrtc/.

[32] Wei J., Wang X., Schuurmans D., Bosma M., Ichter B., Xia F., Chi E., Le Q., Zhou D. (2022), “Chain-of-thought prompting elicits reasoning in large language models”, *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, pp. 24824–24837.

[33] Zhong M., Yin D., Yu T., Zaidi A., Mutuma M., Jha R., Awadallah A. H., Celikyilmaz A., Liu Y., Qiu X., Radev D. (2021), “QMSum: A new benchmark for query-based multi-domain meeting summarization”, *Proceedings of NAACL-HLT 2021*, pp. 5905–5921.